

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Математическая логика и теория формальных языков
Отчёт о выполнении лабораторной работы №1
«Лексический анализатор»

Вариант 35

Студент,

группа 5130201/30102

_____ Путята М. А.

Преподаватель

_____ Востров А. В.

«_____» _____ 2026 г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Математическое описание	4
1.1 Транслятор	4
1.2 Лексический анализатор	4
2 Особенности реализации	9
2.1 Структуры данных	9
2.1.1 Класс лексем	9
2.1.2 Класс эффектов	10
2.1.3 Класс узла диаграммы	11
2.2 Определение параметров лексического анализатора	15
2.3 Функция обработки пользовательского ввода	20
3 Результаты работы программы	24
Список использованной литературы	29

Введение

Компиляция программы – это процесс трансляции исходного кода программы из одного языка в другой (обычно в более низкоуровневый, например в машинный код). Компиляция состоит из следующих основных этапов:

1. **Препроцессинг.** Разрешение зависимостей и обработка макросов (директивы препроцессора).
2. **Лексический анализ.** Выделение лексем в исходном коде программы и получение представления исходной программы как последовательности лексических единиц.
3. **Синтаксический анализ.** Проверка грамматики языка и создание синтаксического дерева, представляющего структуру программы.
4. **Семантический анализ.** Проверка смысловой корректности в контексте правил языка.
5. **Генерация кода.** Преобразование созданного в предыдущих этапах представления программы в программу, написанную на другом языке.

Целью данной работы является разработка программы, выполняющей лексический анализ входного текста в соответствии с заданными требованиями и порождающей таблицу лексем с указанием их типов и значений. Создаваемая программа во время работы должна:

- выделять во входном тексте лексемы;
- определять типы выделяемых лексем;
- проверять длину идентификаторов и строковых констант, ограниченную 15 символами.
- допускать комментарии неограниченной длины;
- выводить сообщения об ошибках, которые могут быть обнаружены на этапе лексического анализа программы.

Вариант 35. Входной язык содержит только латиницу, арифметические выражения, разделённые символом | (вертикальная полоса). Арифметические выражения состоят из идентификаторов, шестнадцатеричных вещественных чисел, знаков операций +, -, *, / и круглых скобок. Шестнадцатеричными числами считать последовательность цифр и символов a, b, c, d, e, f, начинающуюся с «0x» (например, 0x89.ab16 0x45a.0c, 0x0.0af).

1 Математическое описание

1.1 Транслятор

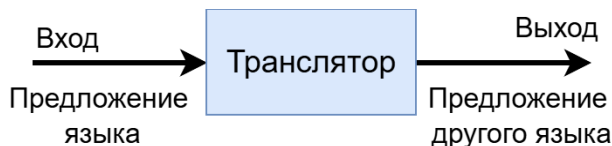


Рис. 1. Транслятор

Транслятор - алгоритм преобразования предложения исходного языка в некоторый выход. Если входной текст является алгоритмическим, то входной текст называют входной программой.

Если задачей транслятора является преобразование в язык низкого уровня, то он называется компилятором, а если выполнение операторов исходного кода – интерпретатором

Процесс синтаксически-ориентированной трансляции состоит из двух этапов:

- Распознавание исходного языка. Построение структуры входного текста на основе порождающей грамматики входного языка.
- Генерация выхода. Использование полученной на предыдущем этапе структуры для создания выхода, удовлетворяющего семантике исходного текста.

1.2 Лексический анализатор

Лексический анализатор проводит лексический анализ (первый этап трансляции), выделяя лексемы входного языка – минимальные единицы языка, которые имеют смысл. В естественном языке лексемами являются слова, а в языке программирования – идентификаторы, служебные символы, константы, операторы и т.д..

Предназначение

Лексический анализ предназначен для определения:

- лексем – подстрок символов, соответствующих распознанным классам лексем.
- классов обнаруживаемых лексем – общее название для категории элементов, обладающих общими свойствами: идентификатор, вещественная константа, строковая константа, операторы и т.д..

Классификация символов входного текста

На вход лексическому анализатору подаются символы, сгруппированные транслятором по определённым категориям. Наиболее типичные группы символов:

- **буква** – множество букв, из 1 и более алфавитов.
- **цифра** – множество символов, относящихся к цифрам, чаще всего от 0 до 9 (для шестнадцатеричной системы счисления от 0 до f).
- **разделитель** – пробел, перевод строки, возврат каретки.
- **игнорируемый** – символы, которые могут присутствовать во входном потоке, но игнорируемые анализатором.
- **запрещённый** – символы, не входящие в алфавит языка.
- **прочие** – символы не принадлежащие к предыдущим категориям.

Формальная модель

Лексический анализатор реализуется с помощью конечного автомата, преобразующего входную цепочку символов в последовательность пар <Тип лексемы, Значение>.

Если заданы:

- $\Sigma = \{\backslash t, \backslash n, " ", (,), *, +, -, /, ., :, =, | \} \cup \{0, \dots, 9\} \cup \{A, \dots, Z\} \cup \{a, \dots, z\}$ – входной алфавит;
- $T = \{ \text{id, arithmetic, assign, left_bracket, right_bracket, const, separator} \}$ – множество типов лексем, где
 - id – идентификатор;
 - assign – знак присваивания «:=»
 - arithmetic – арифметический оператор («+», «-», «*», «/»);
 - left_bracket, right_bracket – левая и правая круглые скобки соответственно;
 - const – вещественная константа, записанная в шестнадцатеричной системе счисления (например 0xa65f);
 - separator – разделитель «|»;
- $D = \{ :=, +, -, *, /, (,), | \} \cup D_{id} \cup D_{const}$ – множество допустимых значений лексем, где

$$D_{id} = \left\{ s \in \Sigma_{id}^+ \left| \begin{array}{l} |s| \in [1, 15], \\ s[0] \in \{A, \dots, Z\} \cup \{a, \dots, z\}, \\ \forall i \in [1, 14] : s[i] \in \Sigma_{id} \end{array} \right. \right\},$$

$$\Sigma_{id} = \{A, \dots, Z\} \cup \{a, \dots, z\} \cup \{0, \dots, 9\}$$

$$- D_{const} = \left\{ s \in \Sigma_{const}^+ \left| \begin{array}{l} |s| \in [3, 15], \\ (s[0] = "0") \wedge (s[1] = "x") \wedge (s[|s| - 1] \neq ".") \\ |\{i \in [3, 14] \mid s[i] = ".\}| \leq 1, \\ \forall i \in [2, 14] : s[i] \in \Sigma_{const} \setminus \{"x"\} \end{array} \right. \right\}$$

$$\Sigma_{const} = \{"x"\} \cup \{a, \dots, z\} \cup \{0, \dots, 9\}$$

То лексический анализатор будет реализовывать отображение $\Sigma^* \longrightarrow (T \times D)^*$, где

- Σ^* – замыкание алфавита входного языка;

- $(T \times D)^*$ – множество пар $\langle \text{Тип лексемы}, \text{Значение} \rangle$.

Синтаксические диаграммы

Для представления поведения конечного автомата (в данном случае лексического анализатора) можно использовать синтаксическую диаграмму, получаемую с помощью преобразования КА. Далее на рисунке 1 представлена синтаксическая диаграмма для лексического анализатора, разрабатываемого в данной работе.

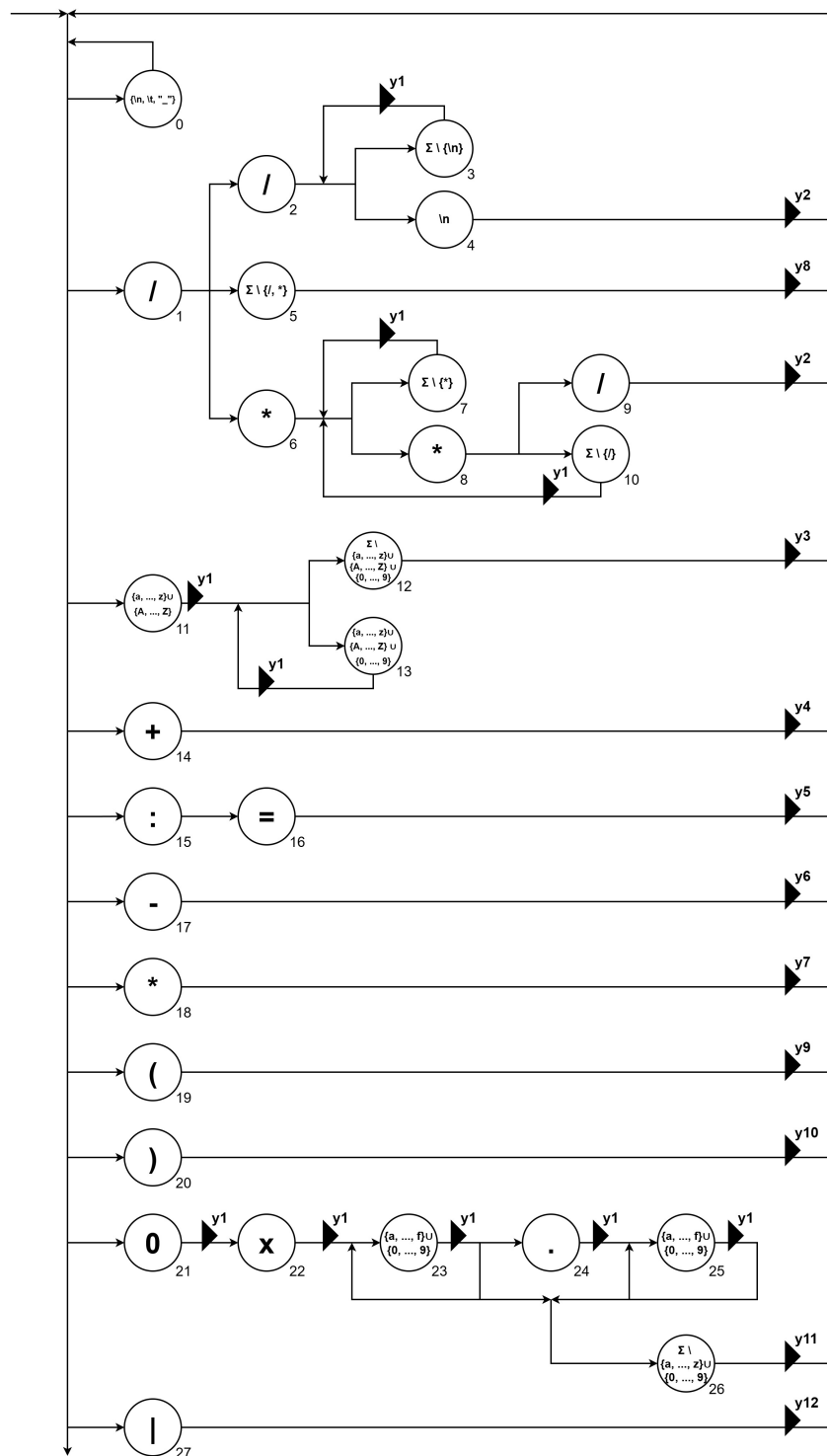


Рис. 2. Синтаксическая диаграмма лексического анализатора

Действия используемые в синтаксической диаграмме:

- $y1$ – записать символ в буфер
- $y2$ – очистить буфер
- $y3$ – выдать лексему $\langle id, значение \rangle$ и очистить буфер

- y4 – выдать лексему $\langle \text{arithmetic}, "+" \rangle$
- y5 – выдать лексему $\langle \text{assign}, " := " \rangle$
- y6 – выдать лексему $\langle \text{arithmetic}, "-" \rangle$
- y7 – выдать лексему $\langle \text{arithmetic}, "*" \rangle$
- y8 – выдать лексему $\langle \text{arithmetic}, "/" \rangle$
- y9 – выдать лексему $\langle \text{left_bracket}, "(" \rangle$
- y10 – выдать лексему $\langle \text{right_bracket}, ")" \rangle$
- y11 – выдать лексему $\langle \text{const}, \text{значение} \rangle$ и очистить буфер
- y12 – выдать лексему $\langle \text{separator}, "|" \rangle$

2 Особенности реализации

2.1 Структуры данных

2.1.1 Класс лексем

С помощью класса `Lexem` в программе реализовано задание типов лексем и хранение выделяемых лексическим анализатором лексем. Также данный класс реализует отслеживание уже добавленных идентификаторов – если добавляемый идентификатор уже был добавлен ранее, то ему присвоится такой же номер, как и предыдущему. Код объявления данного класса представлен на листинге 1.

Листинг 1. Код класса `Lexem`

```
1 class Lexem {
2 private:
3     static map<string, int> type_to_int;
4     static map<int, string> int_to_type;
5     static map<string, string> type_to_rus;
6     static map<string, int> ids;
7     static int count_ids;
8     int type;
9     string token;
10
11 public:
12     static void set_types(initializer_list<pair<string, string>>);
13     static void clr_ids();
14     Lexem(string, string);
15     string get_token();
16     string get_type(bool = false);
17     string get_value();
18 } ;
```

Для задания доступных типов лексем используется статический метод `set_types`. Для каждого типа задаётся два названия: английское (для внутреннего использования) и русское (для вывода в таблицу лексем). Также для каждого добавляемого типа создаётся его числовое представление, а сами названия типов хранятся в словаре, далее в самом классе используются только числовые представления. Данная манипуляция позволяет не хранить для каждой лексемы строку с её типом, что существенно уменьшает количество потребляемой памяти при анализе текстов с большим количеством лексем.

Вход: последовательность пар `<string, string>` – `<Английское название, Русское название>`.

Выход: класс `Lexem` с заданными возможными типами лексем.

Код данного метода представлен на листинге 2.

Листинг 2. Код метода `set_types`

```
1 void Lexem::set_types(initializer_list<pair<string, string>> new_types)
2 {
3     int count_types = 0;
4     for (pair<string, string> t : new_types) {
```

```

4         auto result = Lexem::type_to_int.try_emplace(t.first, count_types)
5         ;
6         if (result.second) {
7             Lexem::int_to_type.emplace(count_types, t.first);
8             Lexem::type_to_rus.emplace(t.first, t.second);
9             count_types++;
10        }
11    }

```

Для создания новой лексемы используется конструктор класса Lexem. Если лексема является идентификатором, то проверяется существует ли уже идентификатор с таким названием, если не существует, то в словарь содержащий номера для всех уникальных идентификаторов добавляется новое значение. Код данного конструктора расположен на листинге 3.

Вход: строка, содержащая тип лексемы, и строка, представляющая токен лексемы из заданного текста.

Выход: новая лексема с заданным токеном, типом, а также значением, если тип – идентификатор.

Листинг 3. Код конструктора класса Effect

```

1 Lexem::Lexem(string type_str, string token) {
2     this->token = token;
3     this->type = Lexem::type_to_int[type_str];
4
5     if (this->type == 0) {
6         auto result = Lexem::ids.try_emplace(this->token, Lexem::count_ids);
7         if (result.second) {
8             Lexem::count_ids += 1;
9         }
10    }
11 }

```

2.1.2 Класс эффектов

Данный класс является обёрткой для функций реализующих выполнение различных действий при переходе между узлами синтаксической диаграммы. Наличие данного класса позволяет использовать вызов объекта как функции для выполнения различных действий. Использование данного класса на данный момент не имеет большой выгоды, но при дальнейшем развитии программы может существенно помочь для реализации более сложных эффектов. Код объявления данного класса представлен на листинге 4.

Листинг 4. Код класса Effect

```

1 class Effect {
2 private:
3     function<void()> func;
4 public:
5     Effect(function<void()>);

```

```

6   void apply();
7 } ;

```

Конструктор класса Effect позволяет создать обёртку вокруг лямбда-функции и в дальнейшем вызывать полученный экземпляр как данную функцию.

Вход: функция, не имеющая аргументов и не возвращающая значения.

Выход: полученная функция обёрнутая в объект класса Effect.

Также для данного класса был переопределён оператор вызова без аргументов, что позволяет запускать выполнение функции эффекта, через вызов самого эффекта.

Вход: функция данного эффекта.

Выход: выполнение функции эффекта.

Коды определения конструктора класса Effect и переопределения оператора можно увидеть на листинге 5.

Листинг 5. Код конструктора класса Effect

```

1 Effect::Effect(function<void()> func) {
2     this->func = func;
3 }
4
5 void Effect::operator()() {
6     this->func();
7 }

```

2.1.3 Класс узла диаграммы

Основой реализации лексического анализатора в разработанной программе является использование конечного автомата, соответствующего созданной синтаксической диаграмме. Для представления в программе диаграммы и её узлов используется один класс Node, конструктор которого даёт возможность добавлять новые узлы в диаграмму, а статические методы и поля класса позволяют использовать сам класс как представление всей синтаксической диаграммы. Далее на листинге 6 представлен код объявления данного класса.

Листинг 6. Код класса Node

```

1 class Node {
2 private:
3     function<bool(char)> match;
4     vector<Node*> next_nodes;
5     vector<Effect*> current_effects;
6     string name;
7
8 public:
9     static Node* start_node;
10    static Node* error_node;
11    static Effect* error_effect;
12    static Node* current_node;
13
14    static deque<char> data;

```

```

15 static char prev_char;
16 static string buffer;
17 static vector<pair<string, string>> errors;
18 static vector<Lexem*> table;
19
20 void set_params(function<bool(char)>, vector<Node*>, vector<Effect*>,
    string);
21 void apply_effects();
22 static bool go_to_next(bool);
23 static pair<vector<Lexem*>, vector<pair<string, string>>> analyze(
    string, bool = false, bool = false);
24 };

```

Назначение полей класса

Обычные поля (для узла диаграммы):

- **match** – функция, возвращающая true, если переход в данный узел по текущему символу возможен;
- **next_nodes** – вектор указателей на следующие узлы диаграммы;
- **current_effects** – вектор эффектов, применяемых при переходе в данный узел;
- **name** – название узла (используется при отладке).

Статические поля (для диаграммы):

- **start_node** – начальный узел диаграммы;
- **error_node** – узел обработчика ошибки, предназначенный для сбора символов токена ошибки;
- **error_effect** – эффект, применяемый при попадании в узел обработчика ошибки;
- **current_node** – текущий узел;
- **data** – двухсторонняя очередь из символов, был выбран такой контейнер для быстрого взятия первого символа текста и более эффективного хранения данных по сравнению с контейнером list;
- **prev_char** – предыдущий взятый символ, используется для правильной обработки ошибки незакрытого многострочного комментария;
- **buffer** – строка представляющая буфер, в котором находятся записанные по мере прохождения узлов значения;
- **errors** – вектор пар строк <Ошибочный токен, Описание ошибки>;
- **table** – вектор указателей на найденные лексемы, хранение указателей позволяет не создавать заново сложные объекты базовых лексем (операторы, скобки, разделитель).

Обычные методы класса (для узла)

В общем случае нельзя задать следующие узлы при создании нового узла, так как, например, при создании петель следующим узлом будет являться данный узел, указатель на который неопределён. Поэтому в определении класса отсутствует конструктор и используется конструктор по умолчанию, который инициализирует нестатические поля из базовыми значениями, а уже определение параметров узла происходит с помощью вызова метода `set_params`.

Метод `set_params` реализует инициализацию параметров узла вместо конструктора. Код данного метода расположен на листинге 7.

Вход: функция, определяющая допустимость перехода в данный узел; следующие узлы; эффекты данного узла; имя узла.

Выход: объект узла синтаксической диаграммы с полностью определёнными полями.

Листинг 7. Код метода `set_params`

```
1 void Node::set_params(function<bool(char)> match, vector<Node*>  
  next_nodes, vector<Effect*> current_effects, string name) {  
2   this->match = match;  
3   this->next_nodes = next_nodes;  
4   this->current_effects = current_effects;  
5   this->name = name;  
6 }
```

После выбора следующего узла, в который должна перейти работа при взятии следующего символа входного текста, происходит выполнение действий (эффектов), определённых для следующего узла. Данный функционал реализован в методе `apply_effects`, который вызывает каждый эффект данного узла.

Вход: узел с определёнными для него эффектами при переходе.

Выход: выполнение эффектов

Код метода `apply_effects` представлен на листинге 8.

Листинг 8. Код метода `apply_effects`

```
1 void Node::apply_effects() {  
2   for (Effect* eff : this->current_effects) {  
3     (*eff)();  
4   }  
5 }
```

Статические методы класса (для всей диаграммы)

После формирования всех узлов, составляющих синтаксическую диаграмму, и задания связей между ними можно начинать работу с самой диаграммой. Для реализации перехода в следующий узел был создан статический метод `go_to_next`. Данный метод при условии наличия символов в цепочке берёт следующий символ, выбирает узел в который будет совершён переход исходя из взятого символа и применяет

эффекты, соответствующие новому узлу. В случае если не был найден подходящий узел, происходит переход в узел обработчика ошибок и применяются его эффекты.

Вход: параметры текущего узла и входная цепочка

Выход: Изменённое состояние синтаксической диаграммы (новый текущий узел), выполненные эффекты следующего узла и булево значение, означающее наличие символов в цепочке.

Код метода `go_to_next` представлен на листинге 9.

Листинг 9. Код метода `go_to_next`

```
1 bool Node::go_to_next() {
2     if (!Node::data.empty()) {
3         char c = Node::data.front();
4         for (Node* next_node : current_node->next_nodes) {
5             if (next_node->match(c)) {
6                 current_node = next_node;
7                 current_node->apply_effects();
8                 return true;
9             }
10        }
11        current_node = error_node;
12        current_node->apply_effects();
13        return true;
14    }
15    return false;
16 }
```

Лексический анализ осуществляется с помощью последовательного взятия следующего символа входной цепочки, перехода в узел, соответствующий взятому символу, и выполнения действий при переходе узел пока не закончатся все символы во входной цепочке. Данный алгоритм реализован в методе `analyze`.

Этот метод производит очистку контекста (таблицы лексем и ошибок) предыдущих лексических анализов при необходимости, последовательно переходит к следующему узлу и выполняет эффекты пока во входной цепочке есть символы, а также выявляет ошибку незаконченного многострочного комментария.

Вход: объект синтаксической диаграммы, входная цепочка символов и флаг очистки контекста.

Выход: таблица лексем и таблица ошибок, полученные в результате лексического анализа.

Код данного метода представлен на листинге 10.

Листинг 10. Код метода `analyze`

```
1 pair<vector<Lexem*>, vector<pair<string, string>>> Node::analyze(string
2     str, bool with_context) {
3
4     Node::data = deque<char>(str.begin(), str.end());
5     Node::current_node = Node::start_node;
6
7     if (!with_context) {
8         table.clear();
9         errors.clear();
10    }
```

```

9      }
10     while (Node::go_to_next(debug)) { };
11     if (!Node::buffer.empty()) {
12         Node::buffer.pop_back();
13         Node::errors.push_back({ Node::buffer, "Многострочный_комментарий_
не_завершён" });
14         Node::buffer.clear();
15     }
16     return { table, errors };
17 }

```

2.2 Определение параметров лексического анализатора

Вся настройка лексического анализатора происходит с помощью работы с методами и полями ранее представленных классов в функции main, только после настройки запуск лексического анализатора считается допустимым.

Далее представлена настройки отдельных компонентов лексического анализатора.

Настройка лексем

Первым делом при настройке лексического анализатора нужно определить входной язык и лексемы, которые он сможет выявлять. Далее на листинге 11 представлено определение символов входного языка, задание допустимых типов лексем и создание базовых лексем, чьи токены постоянны.

Листинг 11. Определение входного языка и лексем

```

1 string alph = "
   abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ1234567890
   +-*/*:=|().\n\t";
2
3 // Задание типов лексем
4 Lexem::set_types({
5     {"id", "Идентификатор"},
6     {"arithmetic", "Арифметический_оператор"},
7     {"assign", "Знак_присваивания"},
8     {"left_bracket", "Левая_скобка"},
9     {"right_bracket", "Правая_скобка"},
10    {"const", "Константа"},
11    {"separator", "Разделитель"}
12 });
13
14 // Создание базовых лексем
15 Lexem* lexem_plus = new Lexem("arithmetic", "+");
16 Lexem* lexem_minus = new Lexem("arithmetic", "-");
17 Lexem* lexem_mult = new Lexem("arithmetic", "*");
18 Lexem* lexem_div = new Lexem("arithmetic", "/");
19 Lexem* lexem_assign = new Lexem("assign", ":=");
20 Lexem* lexem_lt_bracket = new Lexem("left_bracket", "(");

```

```

21 Lexem* lexem_rt_bracket = new Lexem("right_bracket", "");
22 Lexem* lexem_sep = new Lexem("separator", "|");

```

Создание эффектов

После определения лексем определяются все возможные эффекты при переходе в узлы синтаксической диаграммы. Каждый эффект создаётся с помощью конструктора класса Effect с аргументов соответствующим функции выполняющей действия эффекта. Код создания эффектов представлен на листинге 12.

Листинг 12. Код класса Effect

```

1 // удаление первого символа
2 Effect shift([]() {
3     Node::prev_char = Node::data.front();
4     Node::data.pop_front();
5 });
6 // очистка буфера
7 Effect clr_buf([]() { Node::buffer.clear(); });
8 // удаление последнего символа буфера
9 Effect pop_back_buf([]() { Node::buffer.pop_back(); });
10 // добавление ошибки
11 Effect add_error([&cur_group]() {
12     string err_com = get_err_comment(cur_group);
13
14     Node::errors.push_back({ Node::buffer, err_com });
15 });
16 // инкрементирование счётчика символов идентификатора
17 Effect add_char_in_id([&cur_group, &count_chars_identifier]() {
18     if (++count_chars_identifier > 15) {
19         cur_group = Group::id_over;
20     }
21 });
22 // инкрементирование счётчика символов константы
23 Effect add_char_in_number([&cur_group, &count_chars_number]() {
24     if (++count_chars_number > 15) {
25         cur_group = Group::number_over;
26     }
27 });
28
29 // Основные эффекты
30 Effect y1([]() { Node::buffer.push_back(Node::data.front()); });
31 Effect y3([]() { Node::table.push_back(new Lexem("id", Node::buffer)); });
32 Effect y4([&lexem_plus]() { Node::table.push_back(lexem_plus); });
33 Effect y5([&lexem_assign]() { Node::table.push_back(lexem_assign); });
34 Effect y6([&lexem_minus]() { Node::table.push_back(lexem_minus); });
35 Effect y7([&lexem_mult]() { Node::table.push_back(lexem_mult); });
36 Effect y8([&lexem_div]() { Node::table.push_back(lexem_div); });
37 Effect y9([&lexem_lt_bracket]() { Node::table.push_back(lexem_lt_bracket); });
38 Effect y10([&lexem_rt_bracket]() { Node::table.push_back(
    lexem_rt_bracket); });

```



```

39 Effect y11([]() { Node::table.push_back(new Lexem("const", Node::buffer)
    ); });
40 Effect y12([&lexem_sep]() { Node::table.push_back(lexem_sep); });
41
42 // Эффекты для отслеживания группы узлов
43 Effect in_ws([&cur_group, &count_chars_identifier, &count_chars_number
    ]() {
44     cur_group = Group::all;
45     count_chars_identifier = 0;
46     count_chars_number = 0;
47 });
48 Effect in_comment([&cur_group]() { cur_group = Group::comment; });
49 Effect in_mr_comment([&cur_group]() { cur_group = Group::mr_comment; });
50 Effect in_id([&cur_group]() { cur_group = Group::id; });
51 Effect in_assign([&cur_group]() { cur_group = Group::assign; });
52 Effect in_number([&cur_group]() { cur_group = Group::number; });

```

Настройка типов ошибок

Для вывода комментариев к находимым во время лексического анализа ошибкам узлы синтаксической диаграммы были разбиты на следующие группы:

- общая группа;
- обработка однострочного комментария;
- обработка многострочного комментария;
- обработка идентификатора;
- обработка идентификатора превышающего 15 символов;
- обработка оператора присваивания;
- обработка числа;
- обработка числа превышающего 15 символов;

Стоит заметить, что разбиение узлов на данные группы не является статичным, а зависит от самого узла и от текущего количества символов в обрабатываемом идентификаторе или числе. Переход в другую группу при переходе между узлами производится с помощью выполнения соответствующего эффекта для определённой группы.

Для каждой группы были определены комментарии к ошибкам, возникающим в данных группах. Далее в листинге 13 представлен код объявления групп и определение комментариев к ошибкам.

Листинг 13. Создание групп и определение комментариев

```

1 enum class Group {
2     all,
3     comment,
4     mr_comment,
5     id,

```

```

6   id_over,
7   assign,
8   number,
9   number_over
10 };
11
12 string get_err_comment(Group cur_group) {
13     switch (cur_group)
14     {
15     case Group::comment: return "Неизвестные_символы_в_комментарии";
16     case Group::mr_comment: return "Неизвестные_символы_в_многострочном_к
        омментарии";
17     case Group::id: return "Неправильное_название_идентификатора";
18     case Group::id_over: return "Идентификатор_содержит_больше_15_символо
        в";
19     case Group::number: return "Неверный_формат_вещественной_константы";
20     case Group::number_over: return "Вещественная_константа_содержит_боль
        ше_15_символов";
21     default:
22         return "Неизвестная_лексема";
23     }
24 }

```

Создание узлов диаграммы

На листинге 14 можно увидеть создание узлов разработанной синтаксической диаграммы лексического анализатора. Для каждого узла задаётся предикат возможности перехода в данный узел, следующие узлы, эффекты применяемые при переходе в данный узел, и его порядковый номер в качестве имени.

Отдельно было создано два узла для обработки ошибок: узел обработчика ошибки последовательно собирает символы составляющие токен ошибки, а узел сборщика ошибки по текущей группе узлов определяет тип ошибки и возвращает управление в соответствующий следующему символу узел. Также следует обратить внимание, что предикат возможности перехода в узел сборщика ошибки не статичный, а зависит от текущей группы, например, для группы узлов обрабатывающих однострочный комментарий предикатом будет результат проверки принадлежности символа к буквам, цифрам или точке, а для обрабатывающих многострочные комментарии – результат проверки того, что текущий символ равен “/”, а предыдущий – “*”.

Листинг 14. Код создания узлов диаграммы

```

1 // Создание узлов диаграммы
2 vector<Node> nodes(28);
3 vector<Node*> nodes_first({ &nodes[0], &nodes[1], &nodes[11], &nodes
    [14], &nodes[15], &nodes[17], &nodes[18], &nodes[19], &nodes[20], &
    nodes[21], &nodes[27] });
4 nodes[0].set_params([](char c) {return is_ws_char(c); }, nodes_first, {
    &clr_buf, &in_ws, &shift }, "0");
5 nodes[1].set_params(is_slash, { &nodes[2], &nodes[5], &nodes[6] }, { &
    clr_buf, &shift }, "1");
6 nodes[2].set_params(is_slash, { &nodes[3], &nodes[4] }, { &in_comment, &
    shift }, "2");
7 nodes[3].set_params([](char c) {return c != '\n'; }, { &nodes[3], &nodes
    [4] }, { &y1, &shift }, "3");

```

```

8 nodes[4].set_params([](char c) {return c == '\n'; }, nodes_first, { &
  shift }, "4");
9 nodes[5].set_params([](char c) {return c != '/' && c != '*' && in_alph(c
  ); }, nodes_first, { &y8 }, "5");
10 nodes[6].set_params([](char c) {return c == '*'; }, { &nodes[7], &nodes
  [8] }, { &in_mr_comment, &shift }, "6");
11 nodes[7].set_params([](char c) {return c != '*'; }, { &nodes[7], &nodes
  [8] }, { &y1, &shift }, "7");
12 nodes[8].set_params([](char c) {return c == '*'; }, { &nodes[9], &nodes
  [10] }, { &y1, &shift }, "8");
13 nodes[9].set_params(is_slash, nodes_first, { &pop_back_buf, &shift }, "9"
  );
14 nodes[10].set_params([](char c) {return c != '/'; }, { &nodes[7], &nodes
  [8] }, { &in_mr_comment, &y1 }, "10");
15 nodes[11].set_params([](char c) {return 'a' <= c && c <= 'z' || 'A' <= c
  && c <= 'Z'; }, { &nodes[12], &nodes[13] }, { &clr_buf, &in_id, &y1,
  &shift, &add_char_in_id }, "11");
16 nodes[12].set_params([](char c) {return not ('a' <= c && c <= 'z' || 'A'
  <= c && c <= 'Z' || '0' <= c && c <= '9') && in_alph(c); },
  nodes_first, {&y3 }, "12");
17 nodes[13].set_params([&count_chars_identifier](char c) {return ('a' <= c
  && c <= 'z' || 'A' <= c && c <= 'Z' || '0' <= c && c <= '9') &&
  count_chars_identifier <= 15; }, { &nodes[12], &nodes[13] }, { &y1, &
  shift, &add_char_in_id }, "13");
18 nodes[14].set_params([](char c) {return c == '+'; }, nodes_first, { &
  clr_buf, &in_ws, &y4, &shift }, "14");
19 nodes[15].set_params([](char c) {return c == ':'; }, { &nodes[16] }, { &
  clr_buf, &in_assign, &shift }, "15");
20 nodes[16].set_params([](char c) {return c == '='; }, nodes_first, { &y5,
  &in_ws, &shift }, "16");
21 nodes[17].set_params([](char c) {return c == '-'; }, nodes_first, { &
  clr_buf, &in_ws, &y6, &shift }, "17");
22 nodes[18].set_params([](char c) {return c == '*'; }, nodes_first, { &
  clr_buf, &in_ws, &y7, &shift }, "18");
23 nodes[19].set_params([](char c) {return c == '('; }, nodes_first, { &
  clr_buf, &in_ws, &y9, &shift }, "19");
24 nodes[20].set_params([](char c) {return c == ')'; }, nodes_first, { &
  clr_buf, &in_ws, &y10, &shift }, "20");
25 nodes[21].set_params([](char c) {return c == '0'; }, { &nodes[22] }, { &
  clr_buf, &in_number, &y1, &shift, &add_char_in_number }, "21");
26 nodes[22].set_params([](char c) {return c == 'x'; }, { &nodes[23] }, { &
  y1, &shift, &add_char_in_number }, "22");
27 nodes[23].set_params([&count_chars_number](char c) {return ('a' <= c &&
  c <= 'f' || '0' <= c && c <= '9') && count_chars_number <= 15; }, { &
  nodes[23], &nodes[24], &nodes[26] }, { &y1, &shift, &
  add_char_in_number }, "23");
28 nodes[24].set_params([&count_chars_number](char c) {return c == '.' &&
  count_chars_number <= 15; }, { &nodes[25] }, { &y1, &shift, &
  add_char_in_number }, "24");
29 nodes[25].set_params([&count_chars_number](char c) {return ('a' <= c &&
  c <= 'f' || '0' <= c && c <= '9') && count_chars_number <= 15; }, { &
  nodes[25], &nodes[26] }, { &y1, &shift, &add_char_in_number }, "25");
30 nodes[26].set_params([](char c) {return not in_num_chars(c); },
  nodes_first, {&y11, &clr_buf, &in_ws }, "26");
31 nodes[27].set_params([](char c) {return c == '|'; }, nodes_first, { &
  clr_buf, &in_ws, &y12, &shift }, "27");

```

```

32
33 // Узел сборщика ошибки
34 Node error_builder_node;
35 error_builder_node.set_params(
36     [&cur_group](char c) {
37         switch (cur_group)
38         {
39             case Group::id: return not is_id_char(c) && in_alph(c);
40             case Group::id_over: return not is_id_char(c) && in_alph(c);
41             case Group::comment: return c == '\n';
42             case Group::mr_comment:
43                 if (c == '/' && Node::prev_char == '*') {
44                     Node::buffer.pop_back();
45                     return true;
46                 }
47                 return false;
48
49             case Group::number: return !in_num_chars(c);
50             case Group::number_over: return !in_num_chars(c);
51             default: return in_alph(c);
52         }
53     }, nodes_first, { &add_error, &clr_buf, &in_ws }, "error_builder");
54
55 // Узел обработчика ошибки
56 Node::error_node = new Node();
57 vector<Node*> nodes_from_error = nodes_first;
58 nodes_from_error.push_back(Node::error_node);
59 Node::error_node->set_params([], {&return false;}, { &
    error_builder_node}, { &y1, &shift }, "error");

```

2.3 Функция обработки пользовательского ввода

После настройки анализатора его можно запускать с заданием обрабатываемого текста. Функционал обработки пользовательского ввода реализован в функции `input_loop`, которая предлагает пользователю варианты действий и выполняет действие исходя из ввода пользователя.

Данная функция предоставляет пользователю следующие меню:

1. **Меню чтения текста.** Пользователю предоставляется выбор того как ввести код программы для последующей обработки: ручной ввод или ввод из файла. После получения кода программы пользователю предлагается вывести её код и после этого происходит переход в главное меню.
2. **Главное меню.** В данном меню пользователю даётся возможность выполнить следующие действия:
 - провести лексический анализ текста;
 - очистить контекст;
 - вывести код программы;
 - ввести новый код программы;

- ВЫХОД

После выбора пользователя программы выполняет соответствующее действие и возвращает пользователя обратно в главное меню. Под очисткой контекста подразумевается удаление содержимого таблиц лексем и ошибок. Наличие возможности выполнения анализа другого кода без очистки контекста может быть полезно для анализа нескольких исходных кодов как модулей одной программы.

Код определения данной функции представлен на листинге 15.

Листинг 15. Код функции пользовательского ввода

```
1 void input_loop() {
2     cout << "====_Лексический_анализатор_кода_====" << endl;
3     string user_input;
4     string data;
5     while (true) {
6         cout << "\nВыберите_действие:\n\t[1]_Ввести_код_программы\n\t[2]
          _Прочитать_код_программы_из_файла\n\t[exit]_Выход" << endl;
7         getline(cin, user_input);
8         if (user_input == "1") {
9             cout << "\nВведите_код_программы:" << endl;
10            getline(cin, data);
11        }
12        else if (user_input == "2") {
13            bool in_reading = true;
14            bool exit = false;
15            while (in_reading) {
16                string path;
17                cout << "\nВведите_путь_к_файлу_('exit'_чтобы_отменить)
                  :\n";
18                getline(cin, path);
19                if (path == "exit") { exit = true; in_reading = false; }
20                else {
21                    ifstream file(path);
22                    if (file.is_open()) {
23                        data = string(istreambuf_iterator<char>(file),
24                                   istreambuf_iterator<char>());
25                        in_reading = false;
26                        cout << "\033[92mФайл_прочитан.\033[0m\n\tКоличе
27                        ство_строк_-_" << count(data.begin(), data.
28                        end(), '\n') + 1 << "\n\tКоличество_символов_
29                        -_" << utf8_length(data) << endl;
30                        bool in_print_file = true;
31                        while (in_print_file) {
32                            cout << "\nВывести_содержимое_файла?_[Да]/[Н
33                            ет]" << endl;
34                            getline(cin, user_input);
35                            if (user_input == "Да" || user_input == "да"
36                                || user_input == "Д" || user_input == "д
37                                ") {
38                                cout << "\n\033[94m(Начало_программы)
39                                \033[0m\n" << data << "\n\033[94m(Кон
40                                ец_программы)\033[0m" << endl;
41                                in_print_file = false;
42                            }
43                        }
44                    }
45                }
46            }
47        }
48    }
49 }
```

```

33         }
34         else if (user_input == "Нет" || user_input
35                == "нет" || user_input == "Н" ||
36                user_input == "н") {
37             in_print_file = false;
38         }
39         else {
40             cout << "\033[91mВыбор не распознан. Пов
41                 торите попытку.\033[0m" << endl;
42             continue;
43         }
44     }
45     else {
46         cout << "\033[91mВведённый путь не найден. Повто
47             рите попытку.\033[0m" << endl;
48     }
49     if (exit) { continue; };
50 }
51 else if (user_input == "exit"){
52     break;
53 }
54 else{
55     cout << "\033[91mВыбор не распознан. Повторите попытку
56         .\033[0m" << endl;
57     continue;
58 }
59 bool in_output = true;
60 bool exit = false;
61 while (in_output) {
62     cout << "\nВыберите действие:\n\t[1] Провести лексический ан
63         ализ кода программы\n\t[2] Очистить контекст\n\t[3] Вывес
64         ти код программы\n\t[4] Ввести новый код программы\n\t[
65         exit] Выход" << endl;
66     getline(cin, user_input);
67     if (user_input == "1") {
68         print_analysis_result(Node::analyze(data + "\n", false,
69             true));
70     }
71     else if (user_input == "2") {
72         Node::errors.clear();
73         Node::table.clear();
74         Lexem::clr_ids();
75     }
76     else if (user_input == "3") {
77         cout << "\n\033[94m(Начало программы)\033[0m\n" << data
78             << "\n\033[94m(Конец программы)\033[0m" << endl;
79     }
80     else if (user_input == "4") {
81         in_output = false;
82     }
83     else if (user_input == "exit") {
84         exit = true;
85         in_output = false;
86     }
87 }

```

```

79         }
80         else {
81             cout << "\033[91mВыбор не распознан. Повторите попытку
82                 \033[0m" << endl;
83             continue;
84         }
85         if (exit) { break; };
86     }
87     cout << "Выход из программы." << endl;
88 }

```

3 Результаты работы программы

Далее представлены таблица лексем и ошибки при проведение лексического анализа различных текстов.

На рисунке 3 представлен вывод для следующего текста:

```
/*Многострочный  
комменатрий*/  
var1:=0x0.e12fa0|  
    var2 := var1 * (0x15 + 0xf3.91)| \\ обычный комментарий
```

Таблица лексем		
Лексема	Тип лексемы	Значение
var1	Идентификатор	var1 : 0
:=	Знак присваивания	
0x0.e12fa0	Константа	0x0.e12fa0
	Разделитель	
var2	Идентификатор	var2 : 1
:=	Знак присваивания	
var1	Идентификатор	var1 : 0
*	Арифметический оператор	
(	Левая скобка	
0x15	Константа	0x15
+	Арифметический оператор	
0xf3.91	Константа	0xf3.91
)	Правая скобка	
	Разделитель	

Таблица ошибок	
Значение	Тип ошибки

Рис. 3. Результат лексического анализа примера 1

На рисунке 4 представлен вывод для следующего текста, содержащего неправильно записанные числа и незакрытый многострочный комментарий:

```
var1:=0x0.E12fA0 + 0x0.1.fa4  
/*Незакрытый  
многострочный *  
комментарий
```


Таблица лексем		
Лексема	Тип лексемы	Значение
var1	id	var1 : 0
:=	assign	
+	arithmetic	

Таблица ошибок	
Значение	Тип ошибки
0x0.E12fA0	Неверный формат вещественной константы
0x0.1.fa4	Неверный формат вещественной константы
Незакрытый многострочный * комментарий	Многострочный комментарий не завершён

Рис. 4. Результат лексического анализа примера 2

На рисунке 5 представлен вывод для следующего текста, содержащего идентификатор и константу, чьи длины больше 15 символов:

```
longlongvariable := 0x0.ff1
var := 0x123456789.abcdef
```

Таблица лексем		
Лексема	Тип лексемы	Значение
:=	assign	
0x0.ff1	const	0x0.ff1
var	id	var : 0
:=	assign	

Таблица ошибок	
Значение	Тип ошибки
longlongvariable	Идентификатор содержит больше 15 символов
0x123456789.abcdef	Вещественная константа содержит больше 15 символов

Рис. 5. Результат лексического анализа для примера 4

На рисунке 6 представлен вывод для следующего текста, содержащего сочетание символов ”::==” и идентификатор начинающийся с цифры:

```
varNormal ::== 0x57.6f
1varError := varNormal * 0x0.1
```

Таблица лексем		
Лексема	Тип лексемы	Значение
varNormal	id	varNormal : 0
:=	assign	
0x57.6f	const	0x57.6f
:=	assign	
varNormal	id	varNormal : 0
*	arithmetic	
0x0.1	const	0x0.1

Таблица ошибок	
Значение	Тип ошибки
:	Неизвестная лексема
=	Неизвестная лексема
1varError	Неизвестная лексема

Рис. 6. Результат лексического анализа для примера 5

Заключение

В ходе выполнения данной лабораторной работы был разработан лексический анализатор, позволяющий выделять лексемы и обнаруживать ошибки в тексте написанном на заданном языке, который содержит:

- символы латиницы;
- идентификаторы, состоящие из не более чем 15 символов;
- вещественные шестнадцатеричные константы, состоящие из не более чем 15 символов;
- операторы '+', '-', '*', '/', ':=';
- однострочные и многострочные комментарии любой длины;
- символы-разделители арифметических выражений '|';
- круглые скобки.

Для реализации лексического анализатора была построена синтаксическая диаграмма, соответствующая конечному автомату используемого языка.

Полученный лексический анализатор был протестирован на различных примерах кода программ, содержащих и не содержащих ошибок. В ходе тестирования не было выявлено нарушений в его работе – все лексемы были выделены и все ошибки обнаружены.

Достоинства реализации

- Программа написана на языке C++ и в ней используются наиболее эффективные по скорости доступа контейнеры для каждого случая, что позволяет гарантировать высокую скорость обработки больших текстов. Используются следующие контейнеры:
 - Deque – хранение входной цепочки символов. Является наиболее эффективным, так как позволяет брать и удалять элементы в любой позиции за константное время.
 - Vector – хранение узлов КА и действий при переходах. Используется, так как при выборе следующего узла для перехода, производится обход всех следующих узлов, а vector за счёт непрерывности в памяти позволяет выполнять обход максимально быстро.
 - String – хранение буфера символов. Идентификаторы и константы ограничены длиной в 15 символов, а контейнер string как раз оптимизирует хранение строк до 15 символов включительно. Это позволяет не выделять память в куче, а хранить данные последовательно прямо в объекте КА, что уменьшает время доступа к буферу.

- Основой программы является конечный автомат, который за один проход по тексту обрабатывает и лексемы и ошибки, следовательно анализ происходит с линейной сложностью по времени, в отличие от реализации использующей библиотечные регулярные выражения.
- Разработанная структура позволяет легко создавать другие лексические анализаторы с помощью простого задания узлов синтаксической диаграммы и действий при переходах.
- Реализована возможность лексического анализа новых текстов с учётом уже проанализированных в течении одной сессии запуска программы, что позволяет проводить анализ программ, разбитых на несколько модулей.

Недостатки реализации

- Возможно чтение файлов только с кодировкой UTF-8.

Масштабирование

- Реализовать остальные компоненты компилятора: синтаксический и семантический анализаторы, оптимизатор (не обязательно) и генератор кода на машинном языке.
- Добавить графический интерфейс для визуализации ошибок и лексем с помощью подсветки кода.

Список использованной литературы

- [1] Секция «Телематика». — Текст: электронный // tema.spbstu: [сайт]. — URL: <https://tema.spbstu.ru/mathem/> (дата обращения: 06.03.2026).
- [2] Вирт, Н. Построение компиляторов / Н. Вирт. — М.: ДМК Пресс, 2010. — 190 с.
- [3] Ахо, А. В. Компиляторы: принципы, технологии и инструментарий [2-е изд.] / А. В. Ахо, М. С. Лам, Р. Сети, Дж. Д. Ульман ; Пер. с англ. — М. : ООО «И.Д. Вильямс», 2018. — 1184 с.